

Platform Architecture & Demo Readiness

Live championship auctions today. Sport-aware competition operations tomorrow.

SUPER ADMIN

Championship + first admin

CHAMPIONSHIP ADMIN

Rules, teams, players, auction

TEAM CAPTAIN

Scoped live bidding

VIEWER

Read-only live experience

Architecture baseline

Next.js 14 • Spring Boot 3.2 / Java 17 • PostgreSQL 15 • WebSocket/STOMP • JWT in httpOnly cookie

Prepared 22 August 2026 • Based on current repository implementation

Executive verdict & demo gate

Implementation verdict

CODE READY

The requested 10-step journey is supported in code. Super-admin ownership stops after championship/admin provisioning; championship configuration is a separate admin responsibility.

Database gate

ACTION

Run the supplied transactional strict-isolation SQL in `play_neml`, then restart. It adds tenant FKs, versions, race-proof unique keys, indexes and portal mappings.

Validation boundary

VERIFIED BUILD

Backend isolation tests and the frontend production build pass. Live end-to-end certification follows only after the database owner runs the migration.

Presentation journey coverage

1–3

Identity & tenant provisioning

Super admin creates DRAFT championship + first admin; message queued.

4

Competition configuration

Admin sets purse, roster ratio, base price, increment and timer; auction created.

5–7

Participants & access

Admin creates captain/team accounts, queues credentials and adds scoped players.

8–9

Live auction

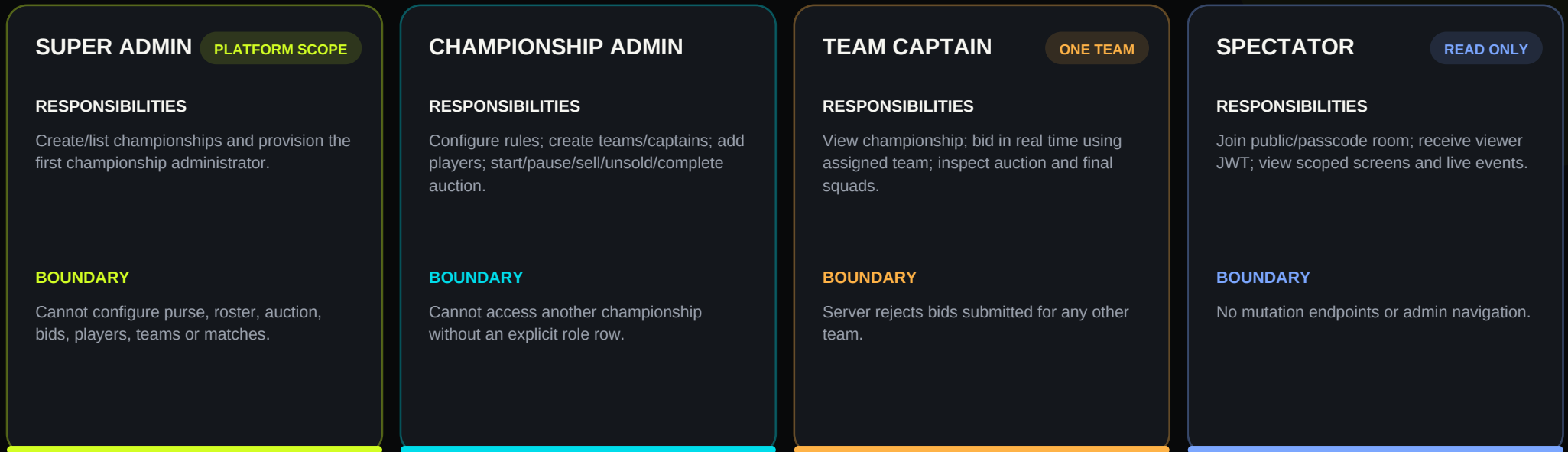
Admin controls the block; captains bid only for their own team; timer and WebSocket sync.

10

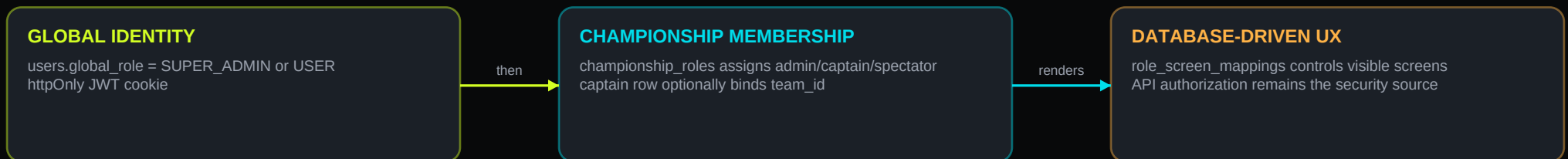
Final squads

Teams screen groups SOLD players, price, gender, skill and remaining purse.

Portal ownership & authorization model

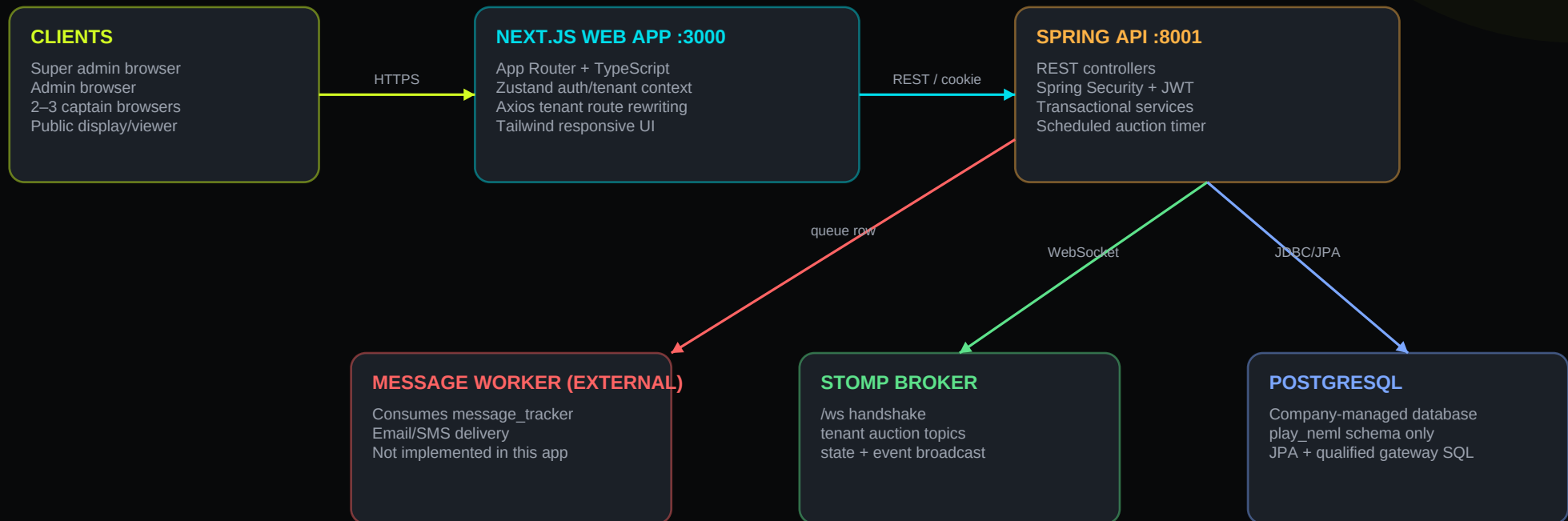


Two-layer authorization



Navigation is usability, not authorization: Spring method security and championship/team predicates protect every sensitive endpoint even if a URL is typed manually.

System context & runtime topology



Deployment boundary

Frontend and backend are local for the presentation. The PostgreSQL host is external/company-owned. Application configuration must set currentSchema=play_neml; code-level native writes explicitly qualify play_neml.message_tracker and cleanup tables.

Backend component architecture

API / SECURITY

AuthController

ChampionshipController

Tenant*Controller

JwtAuthFilter

ChampionshipSecurity

calls

APPLICATION SERVICES

ChampionshipService

TenantAuctionService

MatchService

NavigationService

AnalyticsService

calls

REAL-TIME / INTEGRATION

AuctionBroadcaster

timerSweep scheduler

MessageTrackerGateway

Password generator

calls

PERSISTENCE

Spring Data repositories

JPA entities

JdbcTemplate (qualified SQL)

play_neml schema

Transaction boundary: championship/admin or team/captain/message creation succeeds as one unit. Auction and match events are emitted only after the database transaction commits.

Authentication, tenant isolation & message security

Staff login request



Isolation invariants

- All tenant APIs contain championshipId; repository queries pair child IDs with that championship.
- Captain bid authorization compares requested teamId with championship_roles.team_id.
- Viewer token includes championship_id and cannot mutate resources.
- Per-tab session storage isolates selected championship/auction; WebSocket subscriptions verify both IDs.
- Every manual/native SQL statement targets play_neml explicitly; Hibernate runs in validate-only mode.

Credential notification flow

SENSITIVE

A strong 14-character temporary password is generated. Only its BCrypt hash is stored in users. Delivery fields—email, mobile, userId and temporary password—are serialized into msg_parameters and a PENDING row is inserted in play_neml.message_tracker inside the same transaction.

The creator sees the generated password once. Production hardening: restrict message_tracker readers, encrypt sensitive parameters or use a secret reference, force first-login change, mask logs, and purge delivered credential payloads.

Championship provisioning lifecycle



Atomic provisioning boundary

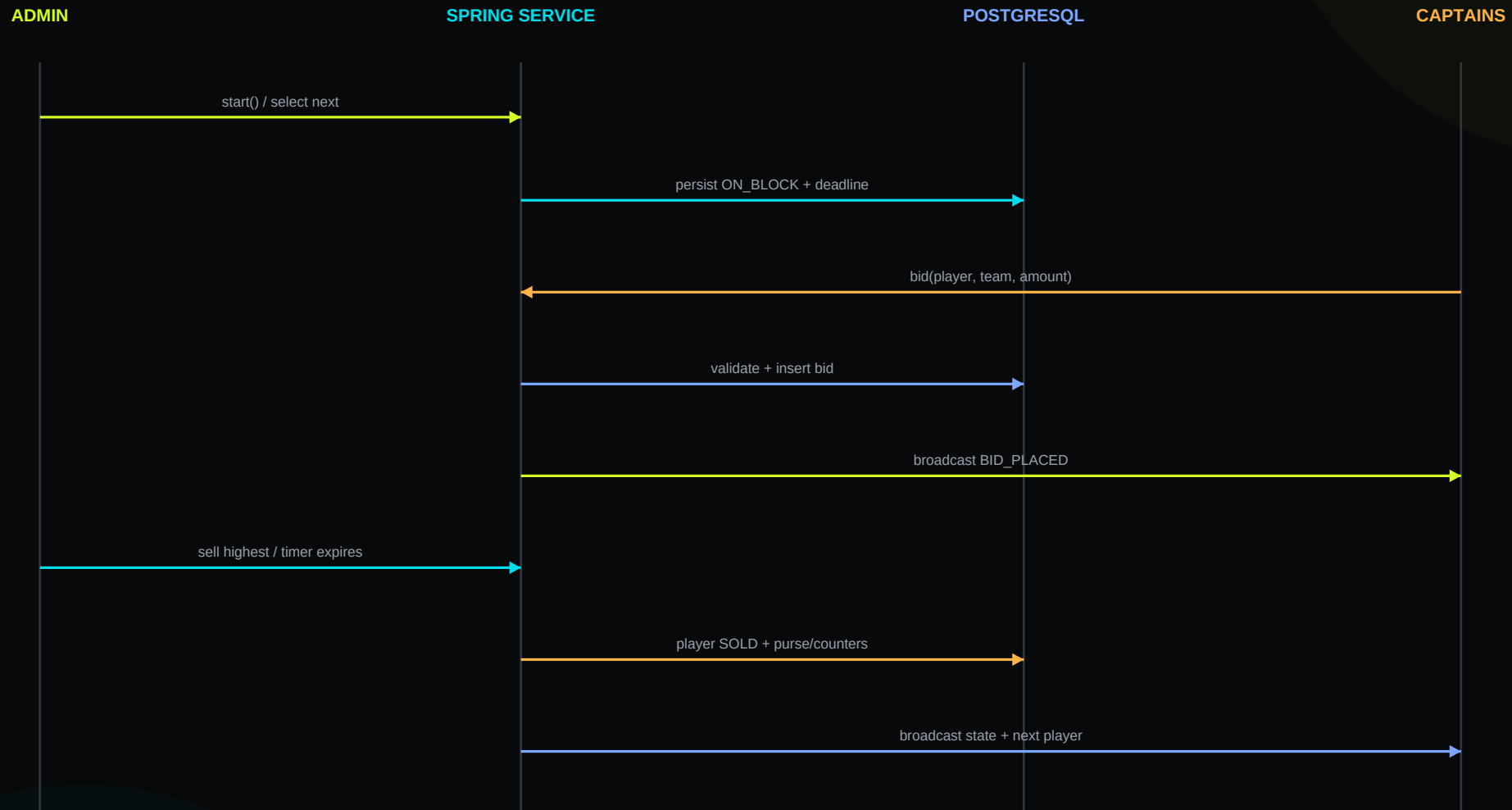
Championship + admin user + championship role + settings + message request commit together. Team + captain user + team role + message request also commit together. A missing message_tracker table or rejected insert rolls back the entire operation.

Configuration lock

Purse totals can be changed before bidding. Once any team has spent funds, the service rejects a purse reconfiguration to avoid corrupting accounting. Rules should be signed off before the client starts the live stage.

Recommended future state machine: DRAFT > CONFIGURED > REGISTRATION_OPEN > AUCTION_READY > AUCTION_LIVE > SQUADS_FINAL > FIXTURES_PUBLISHED > IN_PROGRESS > COMPLETED > ARCHIVED.

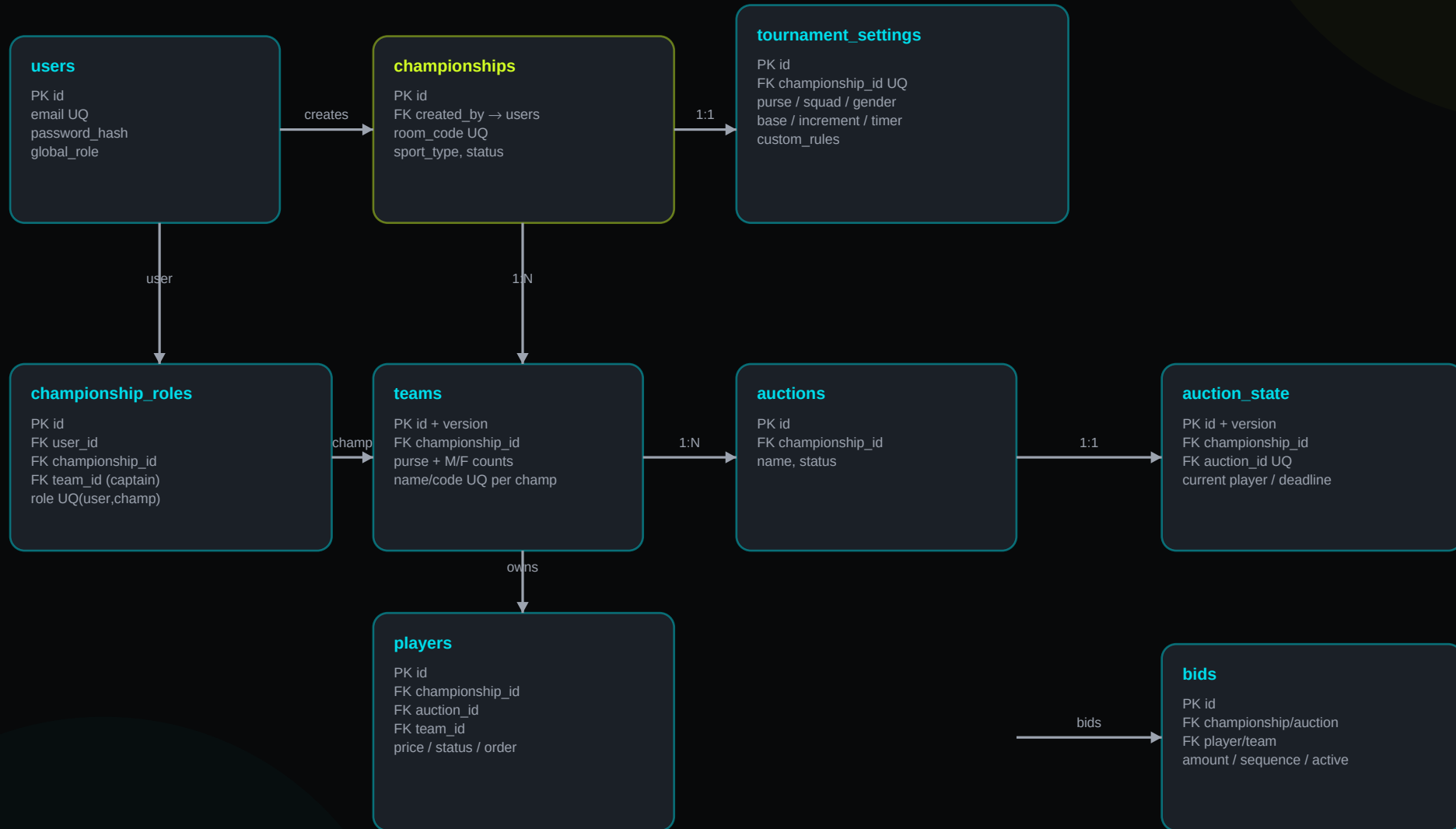
Live auction sequence & business rules



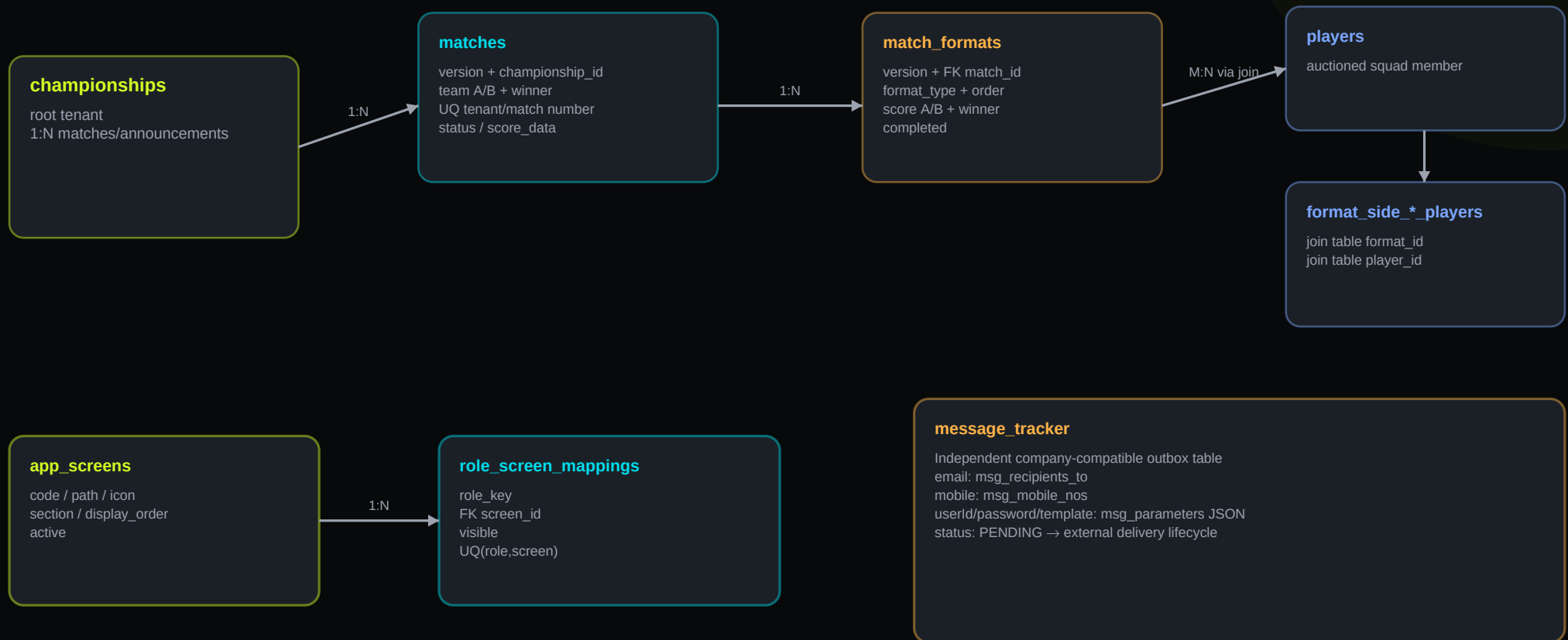
Rules enforced server-side

Minimum increment • own-team authorization • squad capacity • male/female feasibility • purse reserve for remaining slots • auction/player/championship ownership • running-state check

Database architecture I — tenancy, identity & auction



Database architecture II — matches, navigation & messaging



Announcements and seasons now carry championship_id; only the navigation catalogue remains intentionally global. tie_breaker_order is owned through tournament_settings. Composite tenant FKs block cross-championship references in PostgreSQL.

Current data ownership & integrity catalogue

DOMAIN	TABLES	TENANT KEY / OWNER	INTEGRITY
Identity	users, championship_roles	role → championship (+team)	unique email; unique user/champ
Championship	championships, tournament_settings	championship is aggregate root	room code UQ; settings 1:1
Auction	auctions, auction_state, bids	direct championship_id	state 1:1; composite tenant FKs
Roster	teams, players	direct championship_id	team name UQ per championship
Competition	matches, match_formats, side joins	match.championship_id	winner/team/player FKs
Experience	app_screens, role_screen_mappings	global role catalogue	role/screen UQ
Notification	message_tracker	app_id + payload context	outbox lifecycle/status

Schema safety invariant

No code path should create/drop schemas or touch company objects outside play_neml. Production should use a DB account whose grants enforce that boundary even if application code is wrong.

Sport extensibility — separate the engine from the rules

Stable platform kernel

KEEP GENERIC

Identity, roles, championships, teams, players, auction, bids, notifications, WebSocket events and audit are reusable across every sport.

Configurable rule layer

CONFIGURE

Roster constraints, budget logic, player attributes, auction rounds, squad roles, tie breakers and scoring rules should become data-driven definitions.

Sport competition adapter

PLUG IN

Fixture generation, lineup validation, scoring events, standings and sport-specific statistics need versioned strategy implementations.

Examples by championship type

TEAM BADMINTON

gender quotas • singles/doubles formats • player reuse limits • format wins

CRICKET

squad roles • overseas cap • innings/overs • run-rate/tie logic • batting/bowling stats

FOOTBALL

squad/starting XI • formations • substitutions • goals/cards • goal-difference standings

FUTURE SPORT

schema-backed attribute definitions + versioned ruleset + scoring adapter

Avoid one giant nullable team/player/match table. Store universal identity and ownership in normalized core tables; use typed definitions plus sport-specific event/stat tables where relational reporting matters.

Recommended target data model for post-auction growth



Adoption order: introduce sport/ruleset versioning first; migrate current custom_rules; add phases/fixtures; then implement one scoring adapter per sport. Existing auction data remains stable.

Client presentation runbook

T-30 min

Run migration and verification queries; restart backend; confirm `/api/health` and frontend.

T-25 min

Prepare unique admin/captain emails and reachable mobiles. Set auction timer to 60–90 seconds.

T-20 min

Create championship; copy the one-time admin password and confirm the queued message row.

T-15 min

Admin configures auction, creates 3 teams/captains, then adds 6–9 players for a concise demo.

T-10 min

Open separate normal/incognito browser profiles for admin and each captain; test every login.

T-5 min

Verify each captain sees only their team's bid action; verify WebSocket updates across windows.

LIVE

Start, bid, sell every player, open Teams and select each franchise to show final purchases.

Fallback plan

Keep a preconfigured championship and logged-in browser profiles ready. Do not depend on live email/SMS delivery during the client meeting.

Success evidence

Capture `message_tracker` PENDING rows, role-scoped menus, synchronized bids, completed status and final squad/purse cards.

Readiness risks, decisions & next engineering milestones

P0 • BEFORE RESTART

Apply the supplied strict multi-championship SQL in play_neml.

P0 • BEFORE DEMO

Create and rehearse actual admin, captain, team and player records.

P1 • SECURITY

Encrypt or rapidly purge temporary-password payloads; add first-login password change.

P1 • REAL-TIME SCALE

Replace the in-memory STOMP broker with a shared broker before horizontal scaling.

P1 • AUDIT

Persist immutable admin/bid lifecycle events and actor IDs.

P2 • MULTI-SPORT

Implement versioned rule sets and competition adapters described in this document.

Final engineering assessment[GO AFTER GATE](#)

Strict tenant APIs, database ownership constraints, auction locking, scheduled-timer rechecks, tenant WebSocket topics and per-tab context are implemented and build-tested. Run the schema-qualified migration, restart, and rehearse with real demo data.